Chapter 10: Simulation of Computer Systems

Lecture Notes (3 Hours)

Contents: 10.1 Simulation Tools; 10.2 High Level Computer: System Simulation; 10.3 CPU Simulation; 10.4 Memory Simulation; 10.5 Simulation of Computer Networks.

10.1 Simulation Tools

1. Introduction to Simulation

Simulation is the imitation of the operation of a real-world system or process over time by building a *model* and running it on a computer. Instead of building expensive hardware or disturbing a working system, an engineer studies the behaviour of the model, changes its parameters, and observes the results. In computer engineering, simulation is used to design and evaluate processors, memories, buses, operating-system policies and networks *before* they are manufactured or deployed.

Why simulate?

- Real hardware may not yet exist (design-stage evaluation of a new CPU or cache).
- It is cheaper, faster and safer than experimenting on the real system.
- Parameters (cache size, bandwidth, clock, workload) can be varied easily and repeatably.
- Internal details that are impossible to measure on real hardware can be observed.
- Very long or very rare events can be compressed into short run times.

Limitations: a simulation is only as good as its model; simplified assumptions give inaccurate results, detailed simulators run slowly (often 1000× to 100,000× slower than real hardware), and results must always be *verified* and *validated*.

2. Steps in a Simulation Study

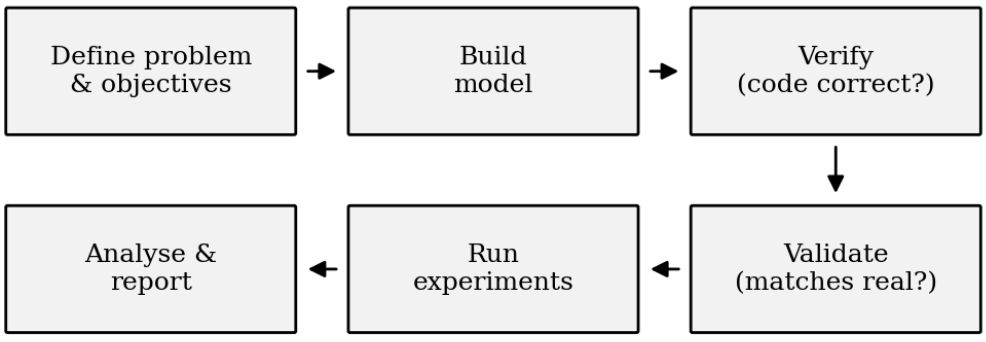


Fig. 10.1: Steps in a simulation study

- **Verification** – checks that the model is implemented correctly ("are we building the model right?").

- **Validation** – checks that the model represents the real system ("are we building the right model?").

3. Types of Simulation

Type	Description	Example use
Discrete-event	State changes only at specific instants (events); a clock jumps from event to event.	Queues, networks, bus arbitration
Continuous	State changes continuously in time, described by differential equations.	Analog circuits, thermal models
Monte Carlo	Uses repeated random sampling to estimate a quantity.	Reliability, performance estimation
Trace-driven	Input is a recorded trace of real events (addresses, instructions, packets).	Cache and memory studies
Execution-driven	The simulator actually executes the program, generating events on the fly.	CPU simulators (gem5)
Full-system	Simulates CPU, memory, devices so an unmodified OS can boot.	QEMU, Simics, gem5 FS mode

4. Common Simulation Tools

Level / domain	Tools	Purpose
Digital logic / hardware design	Logisim, ModelSim, Verilog/VHDL simulators, Xilinx Vivado	Gate- and RTL-level circuit simulation
Circuit level	SPICE, LTspice, Proteus, Multisim	Analog and electronic circuit behaviour
Computer architecture	SimpleScalar, gem5, MARSS, Sniper, Simics	CPU, cache, memory hierarchy, multicore
Instruction set / education	MARS, SPIM, RARS, Ripes	MIPS / RISC-V assembly simulation
Memory	CACTI, DRAMSim / Ramulator, Valgrind-Cachegrind	Cache, DRAM timing and energy
Full system / emulation	QEMU, VirtualBox, Bochs	Run complete operating systems
Networks	NS-2, NS-3, OMNeT++, Cisco Packet Tracer, GNS3, Mininet	Protocol and topology simulation
General / mathematical	MATLAB/Simulink, SimPy (Python), Arena, CloudSim	Discrete-event, system, cloud simulation

5. Features of a Good Simulation Tool

- Accuracy and adjustable level of detail (abstraction).
- Speed and scalability to large models.
- Reusable component libraries and support for standard languages (C/C++, Python, Verilog, SystemC).

- Tracing, waveform viewers and statistics collection.
- Open source, documentation and active community.

10.2 High Level Computer: System Simulation

1. Levels of Abstraction

A computer can be modelled at several levels. Higher levels hide detail, so they are faster to build and run but less precise; lower levels are accurate but slow.

Level	What is modelled	Speed	Accuracy
Circuit / transistor	Voltages, currents, delays	Very slow	Highest
Gate level	Logic gates, flip-flops	Slow	High
Register Transfer Level (RTL)	Registers and data transfers per clock	Moderate	High
Architectural / instruction level	Instructions, registers, memory, caches	Fast	Medium
System level (high level)	CPU, memory, buses, I/O, OS as blocks	Fastest	Lowest (abstract)

2. System-Level (High-Level) Simulation

System simulation models the computer as a set of interacting blocks (processor, memory, bus, disk, I/O devices) and studies overall performance such as throughput, response time and resource utilisation. Components are described by *behaviour and timing* rather than by gates. Languages such as **SystemC** and **Transaction-Level Modelling (TLM)** are widely used, where components exchange whole transactions (e.g. "read 64 bytes") instead of individual signals.

Typical uses: hardware/software co-design, early software development before silicon is ready, bottleneck analysis, capacity planning and design-space exploration.

3. Workload and Performance Metrics

- **Workload** – the jobs applied to the model: real traces, benchmarks (SPEC) or synthetic random generators.
- **Throughput** – jobs completed per unit time.
- **Response time** – time from request to completion.
- **Utilisation** – fraction of time a resource (CPU, disk, bus) is busy.
- **Queue length / waiting time** – how long jobs wait for a resource.

4. Discrete-Event Simulation (DES)

Most system simulators use DES. The simulator keeps a **simulation clock**, a **system state** and a **Future Event List (FEL)** sorted by time. The clock jumps directly to the next event instead of ticking uniformly, which makes simulation efficient.

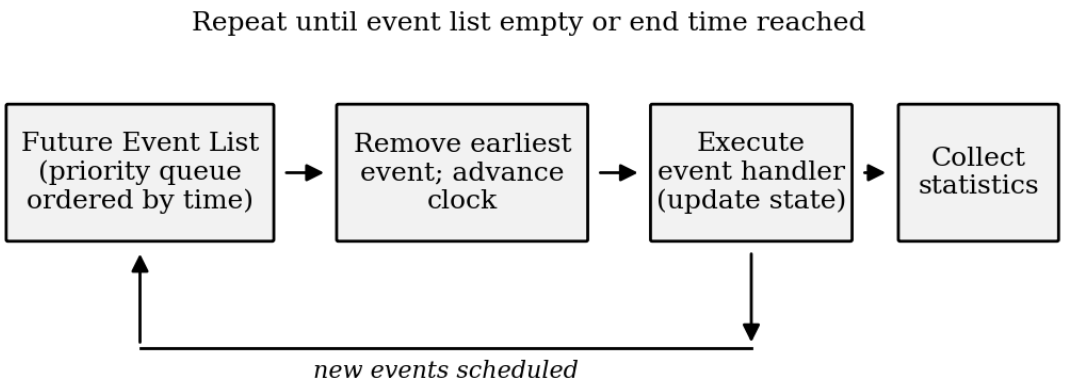


Fig. 10.2: Discrete-event simulation loop

Simplified DES skeleton (Python):

```
import heapq
events = [(0.0, "ARRIVAL")]                 # future event list (priority queue)
clock, queue_len, served = 0.0, 0, 0
while events and clock < 100:
    clock, kind = heapq.heappop(events)     # take earliest event, advance clock
    if kind == "ARRIVAL":
        queue_len += 1
        heapq.heappush(events, (clock + 2.0, "ARRIVAL"))    # next arrival
        heapq.heappush(events, (clock + 1.5, "DEPARTURE"))  # service completes
    else:
        queue_len -= 1; served += 1
print("served:", served)
```

5. Queueing Model of a System (M/M/1)

A simple analytical model for a server (CPU, disk) with random arrivals (rate λ) and service (rate μ), one server, and infinite queue. Utilisation $\rho = \lambda/\mu$ (must be < 1).

Quantity	Formula
Utilisation	$\rho = \lambda / \mu$
Average jobs in system	$L = \rho / (1 - \rho)$
Average time in system	$W = 1 / (\mu - \lambda)$
Average waiting time in queue	$W_q = \rho / (\mu - \lambda)$
Little's law	$L = \lambda \times W$

Example: $\lambda = 8$ jobs/s, $\mu = 10$ jobs/s $\Rightarrow \rho = 0.8$, $L = 4$ jobs, $W = 1/(10-8) = 0.5$ s. Simulation is used when the system is too complex for such closed-form formulas.

10.3 CPU Simulation

A **CPU simulator** is a program that imitates the behaviour of a processor: it reads instructions, updates registers and memory exactly as the real CPU would, and (optionally) counts the time taken. It lets designers test a new instruction set, pipeline or branch predictor without building hardware, and lets students run programs for an architecture they do not own.

1. Structure of a CPU Simulator

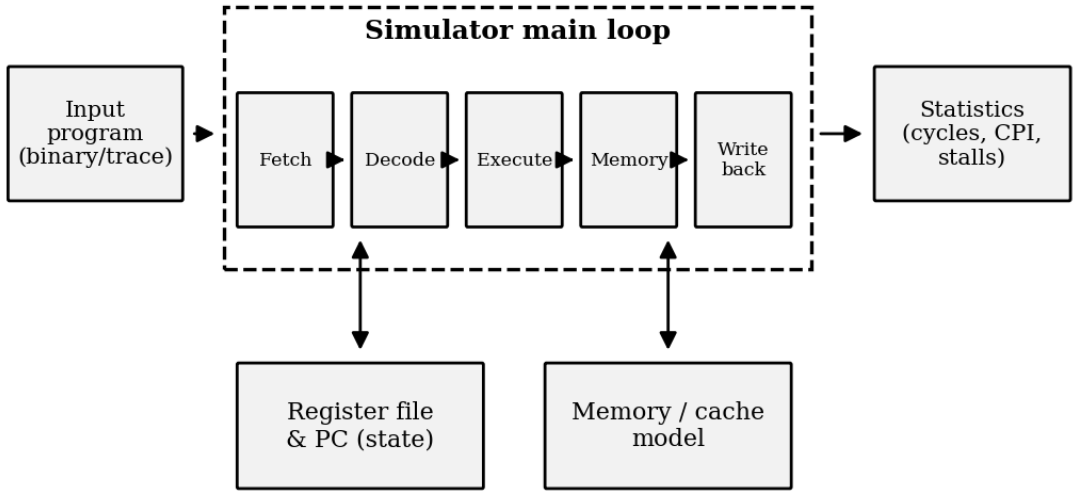


Fig. 10.3: Structure of a CPU simulator

The main loop imitates the **instruction cycle**: **Fetch** the instruction at the address in PC; **Decode** opcode and operands; **Execute** the operation (ALU, branch); access **Memory** for loads/stores; **Write back** the result to a register; then update the PC and statistics.

2. Types of CPU Simulators

Feature	Functional (ISA) simulator	Cycle-accurate simulator	Trace-driven simulator
Goal	Correct program results	Exact timing of every clock cycle	Estimate performance from a recorded trace
Models pipeline?	No	Yes (stages, hazards, stalls)	Sometimes
Speed	Very fast	Slow	Fast
Accuracy of timing	None	Highest	Approximate
Examples	SPIM, MARS, QEMU	gem5 (detailed), SimpleScalar sim-outorder	Cache/branch trace tools

3. Pipeline Simulation

In a five-stage pipeline (IF, ID, EX, MEM, WB) several instructions overlap. A cycle-accurate simulator advances every stage each clock and must detect and model **hazards**:

- **Structural hazard** – two instructions need the same hardware unit at once.
- **Data hazard** – an instruction needs a result not yet produced (resolved by forwarding or stalling).
- **Control hazard** – a branch changes the PC; wrong prediction causes a flush of the pipeline.

4. Performance Metrics

- **CPU time** = Instruction count × CPI × Clock cycle time
- **CPI** (cycles per instruction) = total cycles / instructions executed; with stalls, CPI = ideal CPI + stall cycles per instruction.
- **MIPS** = Instruction count / (Execution time × 10^6)

Example: 10^9 instructions, CPI = 1.5, clock = 2 GHz (0.5 ns) ⇒ CPU time = 10^9 × 1.5 × 0.5 ns = 0.75 s.

5. Example: Tiny Functional Simulator

```
regs = [0] * 8
prog = [("LOADI", 1, 5), ("LOADI", 2, 7), ("ADD", 3, 1, 2), ("HALT",)]
pc, cycles = 0, 0
while True:
    instr = prog[pc]                # FETCH
    op = instr[0]                   # DECODE
    pc += 1; cycles += 1
    if op == "LOADI": regs[instr[1]] = instr[2]                    # EXECUTE
    elif op == "ADD": regs[instr[1]] = regs[instr[2]] + regs[instr[3]]
    elif op == "HALT": break
print(regs[3], cycles)              # prints: 12 4
```

10.4 Memory Simulation

Memory simulation studies how the **memory hierarchy** (registers → cache → main memory → disk) behaves for a given workload. Because memory speed is the main performance limit, designers use simulators to choose cache size, block size, associativity, replacement policy and page size. The usual method is **trace-driven simulation**: a file of memory addresses is fed to a model of the memory, which reports hits, misses and access times.

1. Cache Simulation

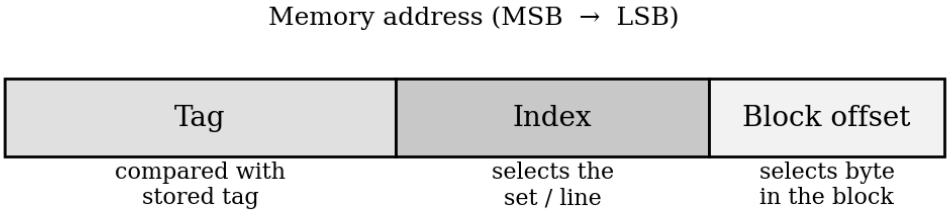


Fig. 10.4: Address fields used by a cache simulator

For each address the simulator extracts the fields, looks in the selected set, compares tags and records a **hit** or **miss**. On a miss the block is loaded, and if the set is full a **replacement policy** chooses the victim.

Mapping	Placement of a block	Advantage	Disadvantage
Direct-mapped	Exactly one line: index = block mod lines	Simple, fast	Conflict misses
Set-associative	Any line inside one set (n-way)	Good balance	More comparators
Fully associative	Any line in the cache	Fewest conflict misses	Costly, slow search

- **Replacement policies:** LRU (least recently used), FIFO, Random.
- **Write policies:** write-through vs write-back; write-allocate vs no-write-allocate.
- **Types of misses (3 Cs):** Compulsory, Capacity, Conflict.

Metrics: Hit ratio = hits / accesses; Miss rate = 1 – hit ratio; **AMAT** = Hit time + Miss rate × Miss penalty. *Example:* hit time 1 ns, miss rate 5%, miss penalty 100 ns ⇒ AMAT = 1 + 0.05 × 100 = 6 ns.

Worked example: trace of block addresses 0, 8, 0, 6, 8 on a cache with 4 blocks.

Cache organisation	Result for each access (M = miss, H = hit)	Misses
Direct-mapped (0, 8 and 6 map to index 0, 0, 2)	M, M, M, M, M	5
2-way set-associative, LRU	M, M, H, M, M	4
Fully associative, LRU	M, M, H, M, H	3

More associativity removes conflict misses, as the simulation shows.

```
def simulate(trace, num_blocks):           # direct-mapped cache simulator
    cache, hits = [None] * num_blocks, 0
    for block in trace:
        idx = block % num_blocks
        if cache[idx] == block: hits += 1  # hit
        else: cache[idx] = block           # miss: load block
    return hits, len(trace) - hits
print(simulate([0, 8, 0, 6, 8], 4))        # (0, 5)
```

2. Virtual Memory (Paging) Simulation

Here the simulator maps **virtual pages** to physical **frames** and counts **page faults**. When memory is full, a page replacement algorithm chooses the page to evict. Input is a *page reference string*.

- **FIFO** – evicts the oldest page (may suffer Belady's anomaly).
- **LRU** – evicts the page unused for the longest time.
- **Optimal (OPT)** – evicts the page not needed for the longest future time; used as a benchmark.

Example: reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 with 3 frames:

Algorithm	Page faults
FIFO	15
LRU	12
Optimal	9

3. Main Memory (DRAM) Simulation

DRAM simulators (e.g. DRAMSim, Ramulator) model banks, rows and columns, and timing parameters such as row activation, column access and precharge. They report access latency, bandwidth and power. A request to an already open row (*row hit*) is much faster than one needing a precharge and new activation (*row conflict*).

10.5 Simulation of Computer Networks

Network simulation models nodes (hosts, routers, switches), links and protocols in software, and studies how packets flow through them. It is a discrete-event simulation in which events are packet arrivals, transmissions, timeouts and drops. It allows testing of routing, congestion control, topologies and new protocols without building a real network.

1. Components of a Network Simulation

- **Nodes** – hosts, routers, switches, access points.
- **Links** – with bandwidth, propagation delay and error rate.
- **Queues** – buffers at interfaces with a discipline (DropTail, RED).
- **Protocols** – TCP, UDP, IP, routing (RIP, OSPF), MAC layer.
- **Traffic generators** – CBR, FTP, HTTP, Poisson sources.
- **Trace and analysis** – log of events used to compute metrics.

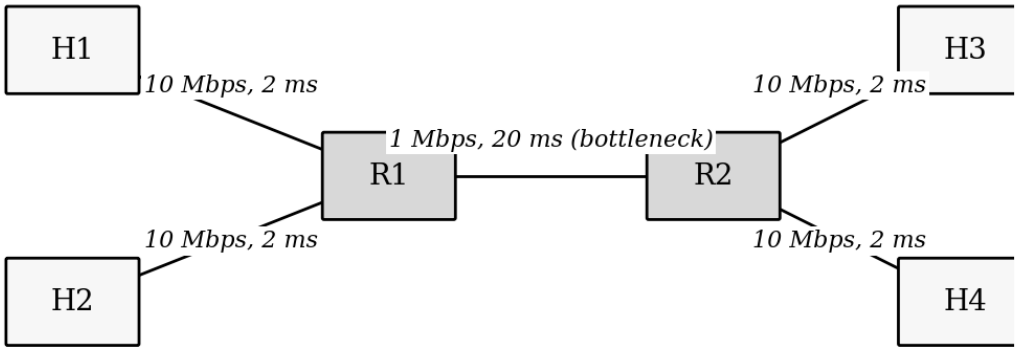


Fig. 10.5: Example simulated topology with a bottleneck link

2. Performance Metrics

Metric	Meaning
Throughput	Data successfully delivered per second (bits/s)
End-to-end delay (latency)	Time from sending to receiving a packet
Jitter	Variation in packet delay
Packet loss ratio	Lost packets / packets sent
Utilisation	Fraction of link capacity in use

3. Network Simulation Tools

Tool	Language / interface	Remarks
NS-2	C++ core, OTcl scripts	Classic research simulator; produces trace files
NS-3	C++ or Python	Successor to NS-2; realistic, actively developed
OMNeT++	C++ with NED, GUI	Modular; INET framework for protocols
Cisco Packet Tracer	Graphical	Education; configure routers and switches
GNS3	Graphical, real IOS images	Emulates real router software
Mininet	Python	Emulates SDN / OpenFlow networks

4. Simulation vs Emulation

Aspect	Simulation	Emulation
Approach	Software model of the network	Runs real software/OS on virtual devices
Realism	Abstract	High (real code and stacks)
Speed / scale	Can run faster or slower than real time; large scale	Real time; limited by hardware
Examples	NS-3, OMNeT++	GNS3, Mininet

5. Example: NS-2 Script (Two Nodes with CBR over UDP)

```
set ns [new Simulator]
set n0 [$ns node]; set n1 [$ns node]
$ns duplex-link $n0 $n1 10Mb 5ms DropTail   ;# bandwidth, delay, queue type
set udp  [new Agent/UDP];  $ns attach-agent $n0 $udp
set sink [new Agent/Null]; $ns attach-agent $n1 $sink
$ns connect $udp $sink
set cbr [new Application/Traffic/CBR]; $cbr attach-agent $udp
$ns at 0.5 "$cbr start"; $ns at 4.5 "$cbr stop"
$ns at 5.0 "exit 0"
$ns run
```

Summary

- Simulation builds a model of a system and studies it by running the model; it must be verified and validated.

- System-level simulation uses high abstraction and discrete-event methods to study throughput and response time.
- CPU simulators imitate the fetch-decode-execute cycle; functional simulators are fast, cycle-accurate ones give timing.
- Memory simulation uses address traces to evaluate caches (hit ratio, AMAT), paging and DRAM.
- Network simulation models packets, links and protocols to measure throughput, delay, jitter and loss.